

Giới thiệu sơ lược về FSDT

Bài trình bày này có tại:

<http://github.com/HytheSkibidiSigma/D-o-n-t-m-laminated-composite-b-ng-FSDT/tree/main>

Dịch thuật & biên soạn: Nguyễn Lê Hy

Ngày 3 tháng 4 năm 2026

Danh mục nội dung

- 1 Nhắc lại những việc đã làm
- 2 Đặt vấn đề
- 3 Tổng quan về FSDT
- 4 Xây dựng và huấn luyện PIN
- 5 Dữ liệu xuất ra được từ Code gốc

Tóm tắt về bài trình bày trước

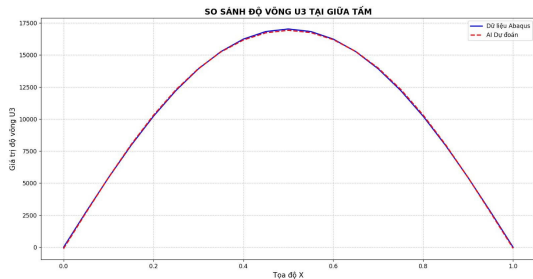
Trong lần thuyết trình hôm trước (06/03/2026), em đã nói qua về các vấn đề như sau:

- Em đã tiến hành thực hiện lại bài sử dụng Khung mạng Nơ-ron Vật lý để dự đoán biến dạng của tấm laminated composite bất đối xứng $[0, 90]$ chịu tải trọng.
- Em đã tiến hành tìm hiểu cách viết code từ Python chuyển sang Abaqus, cộng thêm tham khảo kết quả của Wang [1]

Kết quả thu được

- Em đã nắm rõ hơn và biết cách để từ Python khởi tạo được trên Abaqus.
- Ngoài ra, em cũng đã biết cách xuất file .csv, được dùng để so sánh dữ liệu từ nghiệm giải tích, để so sánh so với kết quả của file .csv, và cuối cùng là so sánh với data train được từ Python.

Kết quả đáng chú ý



Hình: Kết quả mô phỏng

219	0.4	0.5	0	16237.43	-98.0244	9.7353	0
220	0.45	0.5	0	16828.82	-25.2991	2.5176	0
221	0.5	0.5	0	17026.89	0	0	0
222	0.55	0.5	0	16828.82	-25.2991	2.5176	0
223	0.6	0.5	0	16237.43	-98.0244	9.7353	0
224	0.65	0.5	0	15261.77	-207.251	20.6008	0
225	0.7	0.5	0	13917.11	-336.99	33.4715	0
226	0.75	0.5	0	12226.12	-462.737	46.0184	0
227	0.8	0.5	0	10219.44	-560.222	55.6053	0
228	0.85	0.5	0	7937.014	-591.334	58.8037	0
229	0.9	0.5	0	5429.355	-535.015	52.5956	0

Hình: File .csv

Tính chất phức tạp của Tấm Composite xếp lớp

- Cấu trúc bất đối xứng $[0^\circ/90^\circ]$ sinh ra ma trận ghép $B \neq 0$, dẫn đến hiện tượng ghép cặp kéo-uốn phức tạp.

Hạn chế của FEM truyền thống

- Đòi hỏi chia lưới dày đặc, tốn kém tài nguyên và thời gian tính toán.
- Dễ mắc lỗi "khóa cắt" (shear locking) do tỷ số E/G của composite lớn.

Hạn chế của Machine Learning (ML) thuần

- Phụ thuộc hoàn toàn vào Dữ liệu lớn.
- Bản chất "Hộp đen" (Black box), dễ vi phạm các định luật Vật lý khi dự đoán ngoài vùng dữ liệu.

Lý thuyết biến dạng cắt bậc nhất (FSDT)

Lý thuyết tấm Mindlin-Reissner (FSDT) là nền tảng cốt lõi được lựa chọn để phân tích tấm composite trong nghiên cứu này.

Bản chất và giả thiết động học của FSDT

- Pháp tuyến của mặt trung bình trước biến dạng vẫn là một đường thẳng, nhưng không bắt buộc phải vuông góc với mặt trung bình sau khi biến dạng.
- FSDT cho phép các mặt cắt ngang bị trượt đi một góc. Việc xét đến γ_{xz} và γ_{yz} là bắt buộc đối với vật liệu composite do sự chênh lệch rất lớn giữa mô-đun kéo (E) và mô-đun cắt (G).
- Do giả thiết ứng suất cắt phân bố đều theo chiều dày (thay vì hình parabol như thực tế), một hệ số hiệu chỉnh $K_s = 5/6$ được đưa vào để cân bằng năng lượng biến dạng.
- Trường chuyển vị được mô tả hoàn toàn bằng 5 biến độc lập tại mặt trung bình:

$$\mathbf{u} = \{u_0, v_0, w_0, \phi_x, \phi_y\}^T$$

So sánh FSDT với CLPT

Tại sao phải sử dụng FSDT cho tấm Composite?

Vật liệu composite có tỷ số E/G lớn, biến dạng cắt ngang là đáng kể. Việc bỏ qua nó sẽ dẫn đến hiện tượng "khóa cắt" (shear locking) và đánh giá quá cao độ cứng của tấm.

CLPT (Lý thuyết Kirchhoff)

- **Giả thiết:** Pháp tuyến luôn **vuông góc** với mặt trung bình sau biến dạng.
- **Cắt ngang:** Bỏ qua hoàn toàn ($\gamma_{xz} = \gamma_{yz} = 0$).
- **Ứng dụng:** Chỉ đúng với tấm kim loại mỏng. Sai số lớn với tấm composite.
- **Biến độc lập (3):**

$$u_0, v_0, w_0$$

FSDT (Lý thuyết Mindlin)

- **Giả thiết:** Pháp tuyến là đường thẳng nhưng **không bắt buộc vuông góc**.
- **Cắt ngang:** Ứng suất cắt phân bố đều $\rightarrow$ Cần hệ số hiệu chỉnh $K_s = 5/6$.
- **Ứng dụng:** Chính xác cho tấm composite dày và dày trung bình.
- **Biến độc lập (5):**

$$u_0, v_0, w_0, \phi_x, \phi_y$$

Sơ lược về lý thuyết Reissner-Mindlin

Trường chuyển vị theo lý thuyết biến dạng cắt bậc nhất giả thuyết dưới dạng:

$$\begin{aligned}u(x, y, z) &= u_0(x, y) + z\phi_x(x, y) \\v(x, y, z) &= v_0(x, y) + z\phi_y(x, y) \\w(x, y, z) &= w_0(x, y)\end{aligned}\tag{1}$$

Ở đây có sự khác biệt so với bài báo [1] đó là có thêm góc xoay của pháp tuyến mặt trung bình quanh 2 trục x, y . Khác với CLPT, các góc xoay này là những hàm độc lập và không bị ràng buộc bởi đạo hàm của độ võng w_0 .

Mạng Nơ-ron tích hợp vật lý (PINN)

Phương pháp PINN

- Không chỉ học từ dữ liệu mà còn được giám sát bởi các định luật Vật lý.
- Mô hình buộc phải hội tụ về một nghiệm thỏa mãn cơ học thông qua Kỹ thuật vi phân tự động (Automatic Differentiation - AD).

Vi phân tự động (AD) mang lại gì?

- Khắc phục sai số rời rạc hóa của FEM/FDM.
- Tính toán chính xác các đạo hàm không gian tới giới hạn của máy tính.

Phân tách trường biến dạng trong FSDT

Dựa trên giả thiết động học, trường biến dạng được chia thành hai nhóm độc lập. Đây là cơ sở để thiết lập phương trình năng lượng cho mạng nơ-ron.

Biến dạng trong mặt phẳng

Gồm biến dạng màng ϵ^0 và độ cong uốn κ :

$$\epsilon = \epsilon^0 + z\kappa$$

- ϵ^0 phụ thuộc vào u_0, v_0 .
- κ phụ thuộc vào đạo hàm của ϕ_x, ϕ_y .
- **Đặc điểm toán học:** Phân bố **tuyến tính** qua chiều dày z . Mạng AI cần tính đạo hàm không gian bậc 1 của các biến này.

Biến dạng cắt ngang

Đặc trưng tạo nên sự khác biệt của FSDT:

$$\gamma_s = \left\{ \begin{array}{l} \frac{\partial w_0}{\partial y} + \phi_y \\ \frac{\partial w_0}{\partial x} + \phi_x \end{array} \right\}$$

- Phụ thuộc vào cả độ võng w_0 và góc xoay ϕ .
- **Đặc điểm toán học:** Không chứa biến z . Điều này ngụ ý ứng suất cắt bị ép phải phân bố đều.
- **Hệ quả:** Bắt buộc đưa **Hệ số** K_s vào hàm Loss để cân bằng năng lượng.

Phương trình năng lượng

Điểm đột phá của thuật toán không nằm ở việc cung cấp dữ liệu mô phỏng cho AI, mà ở việc biến **Nguyên lý thể năng cực tiểu** thành mục tiêu để AI tự học (Unsupervised Learning).

AI dẫn động bằng dữ liệu

- **Cách học:** Cần hàng ngàn mẫu dữ liệu đầu vào - đầu ra (ví dụ: chạy Abaqus 10.000 lần để lấy độ võng w).
- **Hàm Loss:** So sánh sai số (MSE) giữa dự đoán và dữ liệu mẫu.
- **Nhược điểm:** Nội suy kém khi ra khỏi vùng dữ liệu, hoạt động như một vùng tối.

AI dẫn động bằng Năng lượng

- **Cách học:** Hoàn toàn **KHÔNG** cần dữ liệu FEM để dạy các quy luật bên trong tấm.
- **Hàm Loss:** Mạng nơ-ron tự động tối ưu hóa trọng số để Thể năng toàn phần đạt cực tiểu:

$$\mathcal{L} = \Pi = U(\hat{u}) - W(\hat{w}_0) \rightarrow \min$$

- **Bản chất:** AI tự giải quyết bài toán cơ học, dữ liệu (nếu có) chỉ dùng để ép điều kiện biên.

Tối ưu hóa thể năng toàn phần

Khác với việc giải PDE vi phân truyền thống, mô hình PINN này áp dụng phương pháp Năng lượng (Deep Ritz Method). Mạng nơ-ron tự động cập nhật trọng số để đưa **Thể năng toàn phần Π về trạng thái cực tiểu**.

Thể năng toàn phần trong FSDT: $\Pi = U + V$

1. Năng lượng biến dạng trong (Internal Strain Energy - U): Bao gồm năng lượng màng (U_m), năng lượng uốn (U_b) và đặc biệt là **năng lượng cắt ngang (U_s)** của FSDT:

$$\begin{aligned} U &= U_m + U_b + U_s \\ &= \frac{1}{2} \int_{\Omega} (\epsilon^0{}^T \mathbf{N} + \kappa^T \mathbf{M} + \gamma_s^T \mathbf{Q}) \, dx dy \end{aligned}$$

2. Thể năng của ngoại lực (V):

$$V = - \int_{\Omega} q(x, y) w_0 \, dx dy$$

Tối ưu hóa thể năng toàn phần

Sự ưu việt tuyệt đối của Hàm Loss Năng lượng

Hàm mất mát chính là $\mathcal{L} = U + V$.DOT phá toán học ở đây là các đại lượng biến dạng $\epsilon^0, \kappa, \gamma_s$ **chỉ chứa đạo hàm không gian bậc 1**. Việc loại bỏ hoàn toàn đạo hàm bậc 2 (như trong phương pháp PDEs) giúp AI hội tụ nhanh hơn, mượt hơn và không bị bùng nổ gradient!

Kiến trúc Mạng Nơ-ron Truyền thẳng (FNN)

Cấu trúc Mạng

- **Đầu vào (Input):** Tọa độ không gian (x, y) .
- **Đầu ra (Output):** 5 thành phần chuyển vị $\{u_0, v_0, w_0, \phi_x, \phi_y\}$.
- **Truyền thẳng:** $h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)})$.

Tại sao lại là Tanh thay vì ReLU?

- Hàm ReLU (phổ biến trong AI) có đạo hàm bậc 2 bằng 0 ở mọi nơi.
- Việc giải hệ PDEs của tấm cần đạo hàm không gian $\rightarrow$ Bắt buộc dùng hàm **Hyperbolic Tangent (Tanh)** nhờ tính khả vi liên tục vô hạn.

Bước ngoặt của mô hình: Phương pháp Energy-based PINN

Điểm nghẽn của PINN truyền thống (Dựa trên PDE)

Việc đưa 5 phương trình đạo hàm riêng PDEs (Dạng mạnh) vào hàm Loss đòi hỏi mạng nơ-ron phải tính toán **đạo hàm không gian bậc 2**. Điều này làm tốn tài nguyên GPU, dễ gây bùng nổ gradient.

Giải pháp: Nguyên lý Thế năng Cực tiểu (Deep Ritz Method)

Thay vì giải PDE, bài toán được chuyển thành **Tối ưu hóa không ràng buộc**.

$$\Pi = U - W \rightarrow \min$$

(Với U là Năng lượng biến dạng đàn hồi, W là Công ngoại lực).

Thiết lập hàm mất mát vật lý cho FSDT

Khác với mô hình CLPT (chỉ có U_m, U_b), hàm năng lượng trong FSDT được tích hợp thêm thành phần quan trọng: Năng lượng cắt ngang (U_s).

Công thức tích phân năng lượng biến dạng (U)

$$U = \frac{1}{2} \int_{\Omega} (\epsilon^{0T} A \epsilon^0 + 2\epsilon^{0T} B \kappa + \kappa^T D \kappa + K_s \gamma_s^T A_s \gamma_s) d\Omega$$

- Hệ số tương tác

$$2\epsilon^{0T} B \kappa \quad (2)$$

chứa cả đặc tính ghép kéo-uốn của tấm.

- **Ưu điểm:** Các số hạng biến dạng chỉ chứa đạo hàm không gian bậc 1, giúp hạ bậc bài toán, AI hội tụ cực kỳ mượt mà.

Kỹ thuật ràng buộc cứng cho điều kiện biên

PINN truyền thông dùng hệ số K để ép AI học Điều kiện biên, nhưng rất khó hội tụ và gây nhiễu.

Giải pháp: Ép buộc bằng Toán học thuần túy

Đầu ra thực tế $\hat{u}(x, y)$ được nhân với một hàm khoảng cách $D(x, y)$ tự triệt tiêu tại biên[cite: 3501, 3504]:

$$\hat{w}_0(x, y) = Net_w(x, y) \cdot (x + 1)(x - 1)(y + 1)(y - 1)$$

$$\hat{\phi}_x(x, y) = Net_{\phi_x}(x, y) \cdot (y + 1)(y - 1)$$

→ Khi tọa độ ở viền ($x = \pm 1, y = \pm 1$), hàm đa thức bằng 0 → Chuyển vị chắc chắn bằng 0. AI không cần tốn sức học

Tổng kết về nội dung đã làm tổng bài báo

Ta tóm tắt từ dữ liệu rỗng cho đến dữ liệu cơ học như sau

- ❶ **Khởi tạo:** Lấy N_f mẫu tọa độ (x, y) ngẫu nhiên trong miền Ω .
- ❷ **Dự đoán thô:** Mạng FNN đưa ra 5 biến độc lập (có áp dụng Hard Constraints).
- ❸ **Vi phân tự động:** Tính biến dạng (đạo hàm bậc 1).
- ❹ **Tính Loss:** $\mathcal{L}_{physics} = \Pi = U_m + U_b + U_s + V$.
- ❺ **Tối ưu hóa:** Cập nhật trọng số để đưa Thế năng $\Pi \rightarrow \min$.

Khung mạng Nơ-ron tích hợp kiến thức Vật lý (PINN) trong dự đoán ứng xử uốn của tấm Composite bất đối xứng

Nguyễn Lê Hy^{*1}

¹Khoa Cơ khí, Trường Đại học Bách khoa, Đại học Quốc gia TP.HCM, Việt Nam

Tóm tắt

Việc phân tích kết cấu tấm composite xếp lớp thường phụ thuộc vào các phương pháp số như Phương pháp Phần tử hữu hạn (FEM), vốn đòi hỏi khối lượng tính toán lớn và tài nguyên đáng kể. Nghiên cứu này đề xuất một khung Mạng nơ-ron tích hợp kiến thức vật lý (PINN) nhằm dự đoán trường độ võng của tấm composite $[0^\circ/90^\circ]$ bất đối xứng chịu tải trọng ngang. Bằng cách nhúng trực tiếp lý thuyết biến dạng tấm vào hàm mất mát (loss function) trong quá trình huấn luyện, mạng nơ-ron có khả năng học các định luật vật lý cốt lõi mà không phụ thuộc hoàn toàn vào dữ liệu lớn. Mô hình nhân đầu vào là tọa độ không gian và xuất ra kết quả độ võng. Độ chính xác của khung PINN được kiểm chứng thông qua so sánh với mô phỏng phần tử hữu hạn trên phần mềm Abaqus và lời giải giải tích Navier. Kết quả cho thấy phương pháp học máy đề xuất đạt độ chính xác cao với thời gian tính toán được rút ngắn đáng kể, khẳng định tiềm năng ứng dụng của mô hình thay thế trong kỹ thuật kết cấu.

Từ khóa: Mạng nơ-ron tích hợp kiến thức vật lý (PINN); Tấm composite bất đối xứng; Phương pháp phần tử hữu hạn (FEM); Học sâu; Phân tích uốn.

Hình: Bài báo đang thực hiện

Cấu trúc Mạng Nơ-ron

Mã nguồn sử dụng 3 mạng FNN độc lập cho u_0 , v_0 , w_0 với hàm kích hoạt Tanh. Hàm auto_grad chịu trách nhiệm tính đạo hàm không gian chính xác tuyệt đối.

```
1 class NeuralNetwork(nn.Module):
2     def __init__(self, input_size, output_size, hidden_size=[5, 5, 5, 5, 5], activation='tanh'):
3         super(NeuralNetwork, self).__init__()
4         layers = []
5         layers.append(nn.Linear(input_size, hidden_size[0]))
6         layers.append(nn.Tanh())
7         for i in range(len(hidden_size) - 1):
8             layers.append(nn.Linear(hidden_size[i], hidden_size[i+1]))
9             layers.append(nn.Tanh())
10        layers.append(nn.Linear(hidden_size[-1], output_size))
11        self.network = nn.Sequential(*layers)
12
13    def forward(self, x):
14        return self.network(x)
15
16    def auto_grad(u, x, order=1):
17        if order == 1:
18            return torch.autograd.grad(u, x, torch.ones_like(u), retain_graph=True, create_graph=True)
19            ) [0]
20        return auto_grad(auto_grad(u, x), x, order - 1)
21
```

Trích xuất dữ liệu và in bảng kết quả

Chuẩn hóa số liệu không thứ nguyên và dùng thư viện pandas để xuất định dạng bảng y hệt bài báo.

```
1 W_bar_eval = w_phys.detach().numpy() * 100 * (h)**3 * E2 / (q0 * (2*a)**4)
2 M_xx_bar_eval = M_xx_eval.detach().numpy() / (q0 * (2*a)**2)
3 M_yy_bar_eval = M_yy_eval.detach().numpy() / (q0 * (2*a)**2)
4
5 pinn_preds = [
6     W_bar_eval[0][0], M_xx_bar_eval[0][0], M_yy_bar_eval[0][0],
7     M_xx_bar_eval[1][0], M_yy_bar_eval[2][0]
8 ]
9 ana_sols = analytical_data[name]
10
11 table_rows.append([f"{name} laminated plate", "", "", ""])
12 for i in range(5):
13     pred = pinn_preds[i]
14     ana = ana_sols[i]
15     error = abs(pred - ana) / abs(ana) * 100 if ana != 0 else 0
16     table_rows.append([row_labels[i], f"{pred:.4f}", f"{ana:.4f}", f"{error:.2f}%"])
17
18 df_final = pd.DataFrame(table_rows, columns=["", "Predicted results", "Analytical solutions", "
19     Error"])
20 print("\n" + "="*80)
21 print("Table 1: Comparison of predicted and analytical solutions")
22 print("="*80)
23 print(df_final.to_string(index=False))
24 print("="*80 + "\n")
```

```
*** Using device: cpu

--> Đang huấn luyện mô hình cho tấ: (0°)
Loss:-1.69e-06: 100%|██████████| 10000/10000 [13:28<00:00, 12.37it/s]

--> Đang huấn luyện mô hình cho tấ: (0°/90°)
Loss:-4.63e-06: 100%|██████████| 10000/10000 [13:45<00:00, 12.11it/s]

--> Đang huấn luyện mô hình cho tấ: (0°/90°/0°)
Loss:-8.44e-07: 100%|██████████| 10000/10000 [13:32<00:00, 12.30it/s]

--> Đang huấn luyện mô hình cho tấ: (0°/90°)_2
Loss:-2.20e-06: 100%|██████████| 10000/10000 [13:32<00:00, 12.31it/s]
```

Hình: Phần huấn luyện

So sánh kết quả

Table 1: Comparison of predicted and analytical solutions			
	Predicted results	Analytical solutions	Error
(0°) laminated plate			
w_0 at the center (0, 0)	0.1219	0.1308	6.81%
M_xx at the center (0, 0)	0.0385	0.0439	12.30%
M_yy at the center (0, 0)	0.0006	0.0007	10.84%
M_xx at mid-edge (-a/2, 0)	-0.0882	-0.0870	1.37%
M_yy at mid-edge (0, -b/2)	-0.0072	-0.0112	35.39%
(0°/90°) laminated plate			
w_0 at the center (0, 0)	0.3977	0.3954	0.58%
M_xx at the center (0, 0)	0.0224	0.0239	6.14%
M_yy at the center (0, 0)	0.0276	0.0239	15.64%
M_xx at mid-edge (-a/2, 0)	-0.0602	-0.0553	8.94%
M_yy at mid-edge (0, -b/2)	-0.0454	-0.0553	17.96%
(0°/90°/0°) laminated plate			
w_0 at the center (0, 0)	0.1358	0.1371	0.93%
M_xx at the center (0, 0)	0.0432	0.0444	2.72%
M_yy at the center (0, 0)	0.0012	0.0018	31.40%
M_xx at mid-edge (-a/2, 0)	-0.0892	-0.0881	1.25%
M_yy at mid-edge (0, -b/2)	-0.0142	-0.0157	9.82%
(0°/90°)_2 laminated plate			
w_0 at the center (0, 0)	0.1706	0.1761	3.13%
M_xx at the center (0, 0)	0.0242	0.0249	2.96%
M_yy at the center (0, 0)	0.0192	0.0249	22.88%
M_xx at mid-edge (-a/2, 0)	-0.0525	-0.0565	7.04%
M_yy at mid-edge (0, -b/2)	-0.0580	-0.0565	2.71%

Hình: Kết quả mô phỏng

Table 1
Comparison of predicted and analytical solutions [63] for the deflection and moment resultants of the clamped laminated plates.

(0°) laminated plate	Predicted results	Analytical solutions	Error
$\bar{w}_0$ at the center (0, 0)	0.1324	0.1308	1.22%
$\bar{M}_{xx}$ at the center (0, 0)	0.0430	0.0439	2.03%
$\bar{M}_{yy}$ at the center (0, 0)	0.0007	0.0007	1.48%
$\bar{M}_{xx}$ at mid-edge (-a/2, 0)	-0.0885	-0.0870	1.69%
$\bar{M}_{yy}$ at mid-edge (0, -b/2)	-0.0111	-0.0112	0.98%
(0°/90°) laminated plate			
$\bar{w}_0$ at the center (0, 0)	0.3832	0.3954	3.09%
$\bar{M}_{xx}$ at the center (0, 0)	0.0250	0.0239	4.73%
$\bar{M}_{yy}$ at the center (0, 0)	0.0232	0.0239	2.81%
$\bar{M}_{xx}$ at mid-edge (-a/2, 0)	-0.0580	-0.0553	4.81%
$\bar{M}_{yy}$ at mid-edge (0, -b/2)	-0.0531	-0.0553	4.05 %
(0°/90°/0°) laminated plate			
$\bar{w}_0$ at the center (0, 0)	0.1382	0.1371	0.80%
$\bar{M}_{xx}$ at the center (0, 0)	0.0447	0.0444	0.63%
$\bar{M}_{yy}$ at the center (0, 0)	0.0019	0.0018	3.37%
$\bar{M}_{xx}$ at mid-edge (-a/2, 0)	-0.0893	-0.0881	1.37%
$\bar{M}_{yy}$ at mid-edge (0, -b/2)	-0.0157	-0.0157	0.06%
(0°/90°)_2 laminated plate			
$\bar{w}_0$ at the center (0, 0)	0.1706	0.1761	3.12%
$\bar{M}_{xx}$ at the center (0, 0)	0.0238	0.0249	4.42%
$\bar{M}_{yy}$ at the center (0, 0)	0.0254	0.0249	2.01 %
$\bar{M}_{xx}$ at mid-edge (-a/2, 0)	-0.0552	-0.0565	2.28 %
$\bar{M}_{yy}$ at mid-edge (0, -b/2)	-0.0558	-0.0565	1.22 %

Hình: Kết quả bài báo

Hướng đi tiếp theo

- Tiếp tục phát triển code từ CLPT lên FSDT, vẫn dựa vào code có sẵn trong tài liệu [1]
- Tiếp tục viết báo cá nhân



W. Wang, H.-T. Thai, “A physics-informed neural network framework for laminated composite plates under bending,” *Thin-Walled Structures*, vol. 210, p. 113014, 2025.



Tài liệu nội bộ *Dự án ICMAM Lab*.



Bài báo cá nhân.